Mini-projet: Classification de feuilles d'arbres

Ce notebook est concu pour:

- entrainer plusieurs architectures CNN,
- varier les couches et les hyperparametres,
- comparer les performances,
- produire des elements exploitables dans le compte-rendu.

Objectif vise: precision de validation/test > 75%.

```
import os
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from pathlib import Path
from datetime import datetime

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

from sklearn.metrics import classification_report, confusion_matrix

print('TensorFlow version:', tf.__version__)
```

```
TensorFlow version: 2.21.0
```

```
# ----------------------------
# Configuration generale
# ----------------------------
SEED = 42
IMG_SIZE = (128, 128)
BATCH_SIZE = 32
AUTOTUNE = tf.data.AUTOTUNE

# IMPORTANT:
# DATA_DIR doit pointer vers un dossier avec structure:
# DATA_DIR/
#   classe_1/*.jpg
#   classe_2/*.jpg
#   ...
# Chaque sous-dossier = une classe
```

```
DATA_DIR = "./data_4_classes"  # ex: r"C:\\Users\\ha054\\Documents\\
fdarbre\\leaves_4classes"

# Dossiers candidats automatiques (au cas ou tu as deja place tes
donnees)
CANDIDATE_DIRS = [
    r"C:\Users\ha054\Documents\fdarbre\leaves_4classes",
    r"C:\Users\ha054\Documents\fdarbre\leaves_32classes",
    r"C:\Users\ha054\Documents\fdarbre\dataset",
    r"C:\Users\ha054\Documents\fdarbre\data",
]

np.random.seed(SEED)
tf.random.set_seed(SEED)

for c in CANDIDATE_DIRS:
    if DATA_DIR is None and Path(c).exists():
        DATA_DIR = c

print('DATA_DIR actuel =', DATA_DIR)
if DATA_DIR is None:
    print('-> Definis DATA_DIR manuellement avant de lancer les
cellules suivantes.')
```

```
DATA_DIR actuel = ./data_4_classes
```

```
# ----------------------------
# Chargement des donnees
# ----------------------------
def make_datasets(data_dir, img_size=IMG_SIZE, batch_size=BATCH_SIZE,
seed=SEED):
    train_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset='training',
        seed=seed,
        image_size=img_size,
        batch_size=batch_size,
        label_mode='int',
    )

    val_ds = tf.keras.utils.image_dataset_from_directory(
        data_dir,
        validation_split=0.2,
        subset='validation',
        seed=seed,
        image_size=img_size,
        batch_size=batch_size,
        label_mode='int',
    )
```

```
    class_names = train_ds.class_names

    # Sous-division validation -> validation + test (50/50)
    val_batches = tf.data.experimental.cardinality(val_ds).numpy()
    test_batches = max(1, val_batches // 2)

    test_ds = val_ds.take(test_batches)
    val_ds_final = val_ds.skip(test_batches)

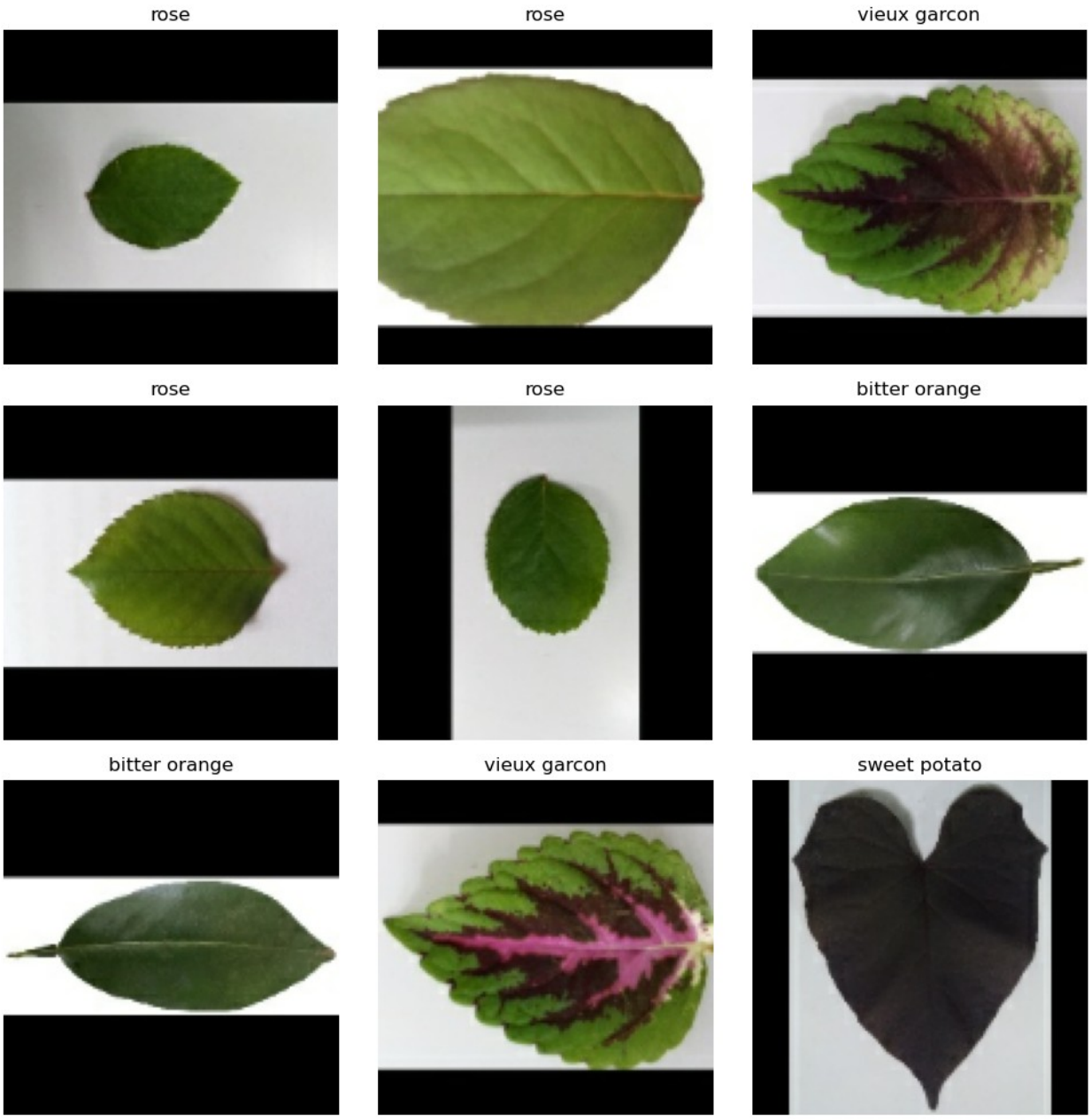
    train_ds = train_ds.cache().shuffle(1000,
seed=seed).prefetch(buffer_size=AUTOTUNE)
    val_ds_final = val_ds_final.cache().prefetch(buffer_size=AUTOTUNE)
    test_ds = test_ds.cache().prefetch(buffer_size=AUTOTUNE)

    return train_ds, val_ds_final, test_ds, class_names

if DATA_DIR is not None:
    train_ds, val_ds, test_ds, class_names = make_datasets(DATA_DIR)
    num_classes = len(class_names)
    print('Classes:', class_names)
    print('Nombre de classes:', num_classes)
else:
    print('DATA_DIR non defini.')
```

```
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Classes: ['bitter orange', 'rose', 'sweet potato', 'vieux garcon']
Nombre de classes: 4
```

```
# Visualisation rapide d'un batch
if DATA_DIR is not None:
    plt.figure(figsize=(10, 10))
    for images, labels in train_ds.take(1):
        for i in range(min(9, len(images))):
            ax = plt.subplot(3, 3, i + 1)
            plt.imshow(images[i].numpy().astype('uint8'))
            plt.title(class_names[labels[i]])
            plt.axis('off')
    plt.tight_layout()
    plt.show()
```




```
# ----------------------------
# Architectures (variation couches)
# ----------------------------
def build_model(variant='A', num_classes=4, input_shape=(128, 128, 3),
dropout=0.3):
    data_aug = keras.Sequential([
        layers.RandomFlip('horizontal'),
        layers.RandomRotation(0.08),
        layers.RandomZoom(0.1),
    ], name='augmentation')
```

```
    inputs = keras.Input(shape=input_shape)
    x = data_aug(inputs)
    x = layers.Rescaling(1.0 / 255)(x)

    if variant == 'A':
        # 2 blocs Conv
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)

    elif variant == 'B':
        # 3 blocs Conv
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(128, 3, activation='relu', padding='same')
(x)
        x = layers.MaxPooling2D()(x)

    elif variant == 'C':
        # 3 blocs Conv + Dropout
        x = layers.Conv2D(32, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Conv2D(128, 3, activation='relu', padding='same')
(x)
        x = layers.MaxPooling2D()(x)
        x = layers.Dropout(dropout)(x)

    else:
        raise ValueError('variant doit etre A, B ou C')

    x = layers.Flatten()(x)
    x = layers.Dense(128, activation='relu')(x)
    if variant == 'C':
        x = layers.Dropout(dropout)(x)
    outputs = layers.Dense(num_classes, activation='softmax')(x)

    return keras.Model(inputs, outputs, name=f'model_{variant}')

# ----------------------------
# Entrainement + comparaison (variation parametres)
# ----------------------------
EXPERIMENTS = [
    # Étude du nombre d'epochs avec une architecture simple
    {'name': 'P1_ep3_A_adam',  'variant': 'A', 'optimizer': 'adam',
'lr': 1e-3, 'batch_size': 32, 'epochs': 3,  'dropout': 0.0},
```

```
    {'name': 'P1_ep5_A_adam',  'variant': 'A', 'optimizer': 'adam',
'lr': 1e-3, 'batch_size': 32, 'epochs': 5,  'dropout': 0.0},
    {'name': 'P1_ep10_A_adam', 'variant': 'A', 'optimizer': 'adam',
'lr': 1e-3, 'batch_size': 32, 'epochs': 10, 'dropout': 0.0},

    # Étude de la profondeur du réseau
    {'name': 'P1_2blocs_A_adam', 'variant': 'A', 'optimizer': 'adam',
'lr': 1e-3, 'batch_size': 32, 'epochs': 5, 'dropout': 0.0},
    {'name': 'P1_3blocs_B_adam', 'variant': 'B', 'optimizer': 'adam',
'lr': 1e-3, 'batch_size': 32, 'epochs': 5, 'dropout': 0.0},

    # Étude du dropout
    {'name': 'P1_3blocs_C_dropout03', 'variant': 'C', 'optimizer':
'adam', 'lr': 1e-3, 'batch_size': 32, 'epochs': 5, 'dropout': 0.3},

    # Étude de l'optimiseur
    {'name': 'P1_A_rmsprop', 'variant': 'A', 'optimizer': 'rmsprop',
'lr': 1e-3, 'batch_size': 32, 'epochs': 5, 'dropout': 0.0},
]


def get_optimizer(name, lr):
    if name.lower() == 'adam':
        return keras.optimizers.Adam(learning_rate=lr)
    if name.lower() == 'rmsprop':
        return keras.optimizers.RMSprop(learning_rate=lr)
    raise ValueError(f'Optimiseur inconnu: {name}')


def train_one_experiment(exp, data_dir):
    train_ds_local, val_ds_local, test_ds_local, class_names_local =
make_datasets(
        data_dir,
        img_size=IMG_SIZE,
        batch_size=exp['batch_size'],
        seed=SEED,
    )

    model = build_model(
        variant=exp['variant'],
        num_classes=len(class_names_local),
        input_shape=(*IMG_SIZE, 3),
        dropout=exp['dropout'],
    )

    optimizer = get_optimizer(exp['optimizer'], exp['lr'])
    model.compile(
        optimizer=optimizer,
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy'],
```

```
    )

    callbacks = [
        keras.callbacks.EarlyStopping(monitor='val_accuracy',
patience=5, restore_best_weights=True)
    ]

    history = model.fit(
        train_ds_local,
        validation_data=val_ds_local,
        epochs=exp['epochs'],
        callbacks=callbacks,
        verbose=1,
    )

    test_loss, test_acc = model.evaluate(test_ds_local, verbose=0)

    best_val_acc = float(np.max(history.history['val_accuracy']))
    best_train_acc = float(np.max(history.history['accuracy']))

    result = {
        'name': exp['name'],
        'variant': exp['variant'],
        'optimizer': exp['optimizer'],
        'lr': exp['lr'],
        'batch_size': exp['batch_size'],
        'epochs_target': exp['epochs'],
        'epochs_ran': len(history.history['loss']),
        'best_train_acc': best_train_acc,
        'best_val_acc': best_val_acc,
        'test_acc': float(test_acc),
        'test_loss': float(test_loss),
    }

    return model, history, result, class_names_local, test_ds_local


all_results = []
all_histories = {}
all_models = {}

if DATA_DIR is not None:
    for exp in EXPERIMENTS:
        print('\n' + '=' * 80)
        print('Lancement:', exp['name'])
        model, history, result, class_names, test_ds =
train_one_experiment(exp, DATA_DIR)
        all_results.append(result)
        all_histories[exp['name']] = history.history
        all_models[exp['name']] = model
```

```
    results_df =
pd.DataFrame(all_results).sort_values(by='best_val_acc',
ascending=False)
    display(results_df)

    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
    csv_name = f'resultats_cnn_{timestamp}.csv'
    json_name = f'resultats_cnn_{timestamp}.json'
    hist_name = f'historiques_cnn_{timestamp}.json'

    results_df.to_csv(csv_name, index=False)
    with open(json_name, 'w', encoding='utf-8') as f:
        json.dump(all_results, f, indent=2)
    with open(hist_name, 'w', encoding='utf-8') as f:
        json.dump(all_histories, f, indent=2)

    print(f'Fichiers sauves: {csv_name}, {json_name}, {hist_name}')
else:
    print('Definis DATA_DIR avant execution.')
```

```
==========================================================================
==========
Lancement: P1_ep3_A_adam
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/3
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 115ms/step - accuracy: 0.4375 - loss:
1.9124 - val_accuracy: 0.7667 - val_loss: 0.7676
Epoch 2/3
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 103ms/step - accuracy: 0.8306 - loss:
0.4739 - val_accuracy: 0.9167 - val_loss: 0.3205
Epoch 3/3
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 102ms/step - accuracy: 0.9254 - loss:
0.2254 - val_accuracy: 0.9833 - val_loss: 0.1677

==========================================================================
==========
Lancement: P1_ep5_A_adam
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 112ms/step - accuracy: 0.4133 - loss:
1.7925 - val_accuracy: 0.6500 - val_loss: 0.8478
Epoch 2/5
```

```
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 101ms/step - accuracy: 0.7722 - loss:
0.6190 - val_accuracy: 0.8833 - val_loss: 0.4320
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 98ms/step - accuracy: 0.8911 - loss:
0.3233 - val_accuracy: 0.9667 - val_loss: 0.1756
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 102ms/step - accuracy: 0.9456 - loss:
0.1603 - val_accuracy: 0.9667 - val_loss: 0.1351
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 103ms/step - accuracy: 0.9657 - loss:
0.1061 - val_accuracy: 0.9833 - val_loss: 0.0533

==================================================================
==========
Lancement: P1_ep10_A_adam
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 110ms/step - accuracy: 0.4798 - loss:
1.7163 - val_accuracy: 0.6833 - val_loss: 0.8203
Epoch 2/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 99ms/step - accuracy: 0.8508 - loss:
0.4874 - val_accuracy: 0.9333 - val_loss: 0.2414
Epoch 3/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 99ms/step - accuracy: 0.9375 - loss:
0.1896 - val_accuracy: 0.9667 - val_loss: 0.1736
Epoch 4/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 100ms/step - accuracy: 0.9556 - loss:
0.1320 - val_accuracy: 0.9667 - val_loss: 0.1315
Epoch 5/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 96ms/step - accuracy: 0.9637 - loss:
0.1070 - val_accuracy: 0.9500 - val_loss: 0.0913
Epoch 6/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 97ms/step - accuracy: 0.9758 - loss:
0.0690 - val_accuracy: 1.0000 - val_loss: 0.0206
Epoch 7/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 97ms/step - accuracy: 0.9839 - loss:
0.0449 - val_accuracy: 0.9833 - val_loss: 0.0302
Epoch 8/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 96ms/step - accuracy: 0.9758 - loss:
0.0660 - val_accuracy: 0.9833 - val_loss: 0.0297
Epoch 9/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 96ms/step - accuracy: 0.9879 - loss:
0.0504 - val_accuracy: 1.0000 - val_loss: 0.0304
Epoch 10/10
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 101ms/step - accuracy: 0.9798 - loss:
0.0584 - val_accuracy: 1.0000 - val_loss: 0.0121
```

```
==========
Lancement: P1_2blocs_A_adam
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 107ms/step - accuracy: 0.6089 - loss:
0.9790 - val_accuracy: 0.8500 - val_loss: 0.3956
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 98ms/step - accuracy: 0.9214 - loss:
0.2534 - val_accuracy: 0.8833 - val_loss: 0.2834
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 96ms/step - accuracy: 0.9637 - loss:
0.1105 - val_accuracy: 0.9333 - val_loss: 0.1232
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 97ms/step - accuracy: 0.9677 - loss:
0.0731 - val_accuracy: 0.9833 - val_loss: 0.0524
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 100ms/step - accuracy: 0.9798 - loss:
0.0726 - val_accuracy: 1.0000 - val_loss: 0.0209

==========
Lancement: P1_3blocs_B_adam
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 105ms/step - accuracy: 0.5060 - loss:
1.1585 - val_accuracy: 0.7667 - val_loss: 0.5818
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.8266 - loss:
0.4240 - val_accuracy: 0.9667 - val_loss: 0.2429
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.9395 - loss:
0.2068 - val_accuracy: 0.9667 - val_loss: 0.1512
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.9597 - loss:
0.1078 - val_accuracy: 0.9667 - val_loss: 0.0602
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 90ms/step - accuracy: 0.9778 - loss:
0.0631 - val_accuracy: 0.9833 - val_loss: 0.0366

==========
Lancement: P1_3blocs_C_dropout03
```

```
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 104ms/step - accuracy: 0.4940 - loss:
1.1089 - val_accuracy: 0.7167 - val_loss: 0.5650
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 93ms/step - accuracy: 0.8306 - loss:
0.4594 - val_accuracy: 0.9667 - val_loss: 0.1899
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 93ms/step - accuracy: 0.9173 - loss:
0.2207 - val_accuracy: 0.9667 - val_loss: 0.0959
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 94ms/step - accuracy: 0.9536 - loss:
0.1263 - val_accuracy: 1.0000 - val_loss: 0.0147
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.9677 - loss:
0.0996 - val_accuracy: 0.9833 - val_loss: 0.0370

==========================================================================
==========
Lancement: P1_A_rmsprop
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
Epoch 1/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 3s 104ms/step - accuracy: 0.4516 - loss:
1.8075 - val_accuracy: 0.5667 - val_loss: 0.9802
Epoch 2/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 92ms/step - accuracy: 0.6714 - loss:
0.8339 - val_accuracy: 0.7333 - val_loss: 0.6696
Epoch 3/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 2s 93ms/step - accuracy: 0.8669 - loss:
0.4419 - val_accuracy: 0.8833 - val_loss: 0.3122
Epoch 4/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 90ms/step - accuracy: 0.9133 - loss:
0.2303 - val_accuracy: 0.6833 - val_loss: 1.3726
Epoch 5/5
16/16 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.9153 - loss:
0.3347 - val_accuracy: 0.9833 - val_loss: 0.1142

                     name variant optimizer     lr  batch_size
epochs_target  \
2          P1_ep10_A_adam       A      adam  0.001          32
10
5   P1_3blocs_C_dropout03       C      adam  0.001          32
5
3        P1_2blocs_A_adam       A      adam  0.001          32
```

```
5
1          P1_ep5_A_adam       A      adam  0.001           32
5
0          P1_ep3_A_adam       A      adam  0.001           32
3
4       P1_3blocs_B_adam       B      adam  0.001           32
5
6           P1_A_rmsprop       A   rmsprop  0.001           32
5

   epochs_ran  best_train_acc  best_val_acc  test_acc  test_loss
2          10        0.987903      1.000000  0.984375   0.028455
5           5        0.967742      1.000000  1.000000   0.016888
3           5        0.979839      1.000000  1.000000   0.012366
1           5        0.965726      0.983333  1.000000   0.026950
0           3        0.925403      0.983333  1.000000   0.074630
4           5        0.977823      0.983333  0.984375   0.032342
6           5        0.915323      0.983333  1.000000   0.043528

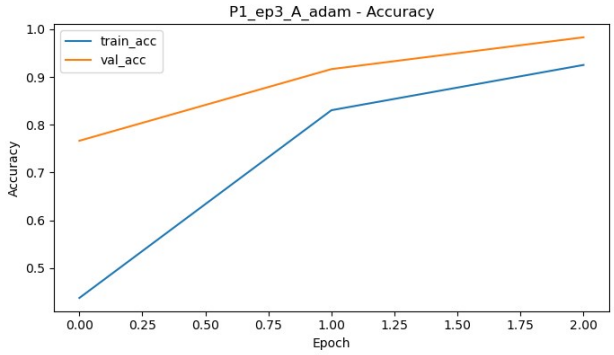
Fichiers sauves: resultats_cnn_20260507_233313.csv,
resultats_cnn_20260507_233313.json,
historiques_cnn_20260507_233313.json
```

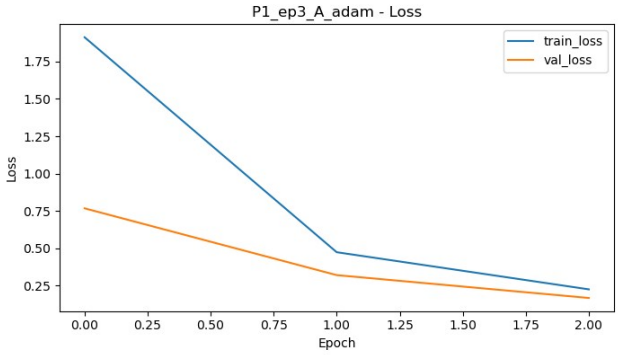
```python
# Courbes d'apprentissage des experiences
if DATA_DIR is not None and len(all_histories) > 0:
    n = len(all_histories)
    plt.figure(figsize=(14, 4 * n))

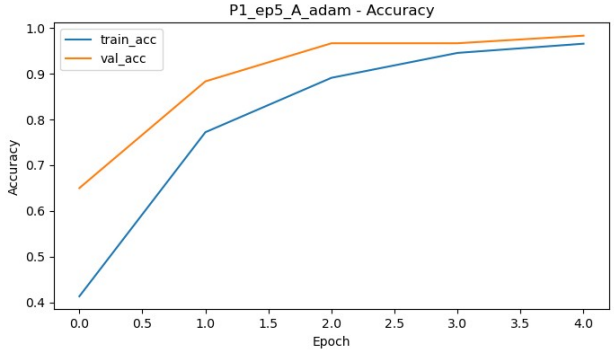
    for i, (name, hist) in enumerate(all_histories.items(), start=1):
        plt.subplot(n, 2, 2 * i - 1)
        plt.plot(hist['accuracy'], label='train_acc')
        plt.plot(hist['val_accuracy'], label='val_acc')
        plt.title(f'{name} - Accuracy')
        plt.xlabel('Epoch')
        plt.ylabel('Accuracy')
        plt.legend()

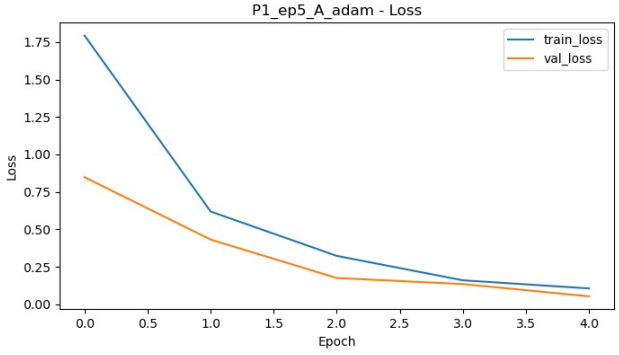
        plt.subplot(n, 2, 2 * i)
        plt.plot(hist['loss'], label='train_loss')
        plt.plot(hist['val_loss'], label='val_loss')
        plt.title(f'{name} - Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.legend()

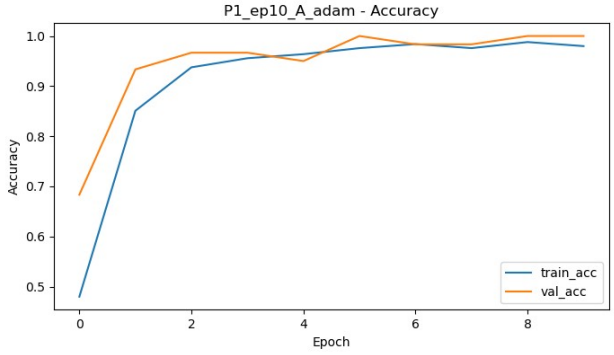
    plt.tight_layout()
    plt.show()
```

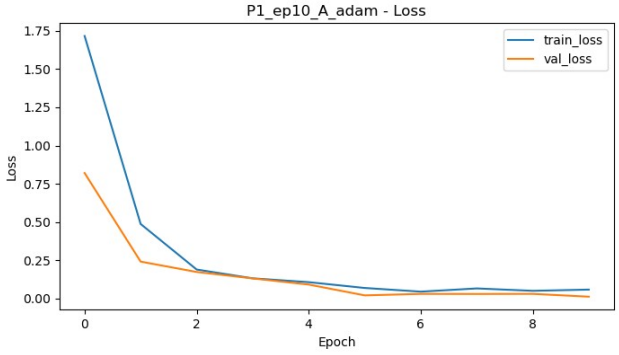

P1_ep3_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch

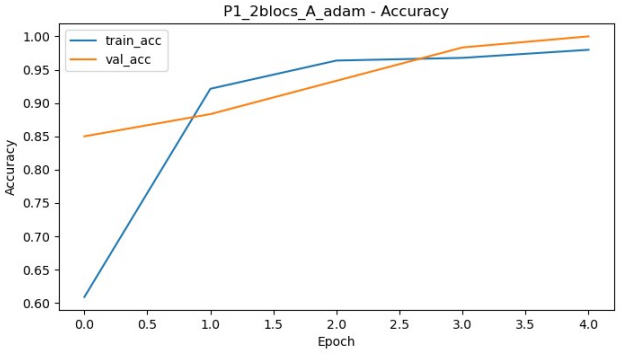

P1_ep3_A_adam - Loss
train_loss
val_loss
Loss
Epoch

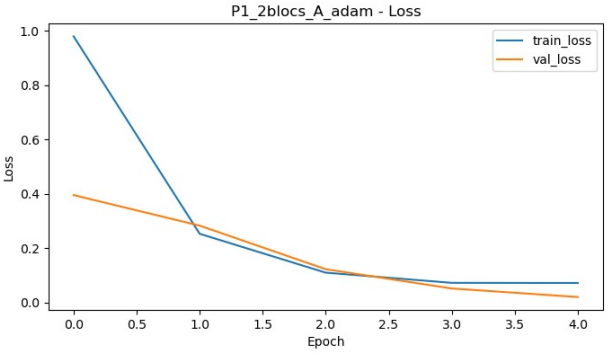

P1_ep5_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch

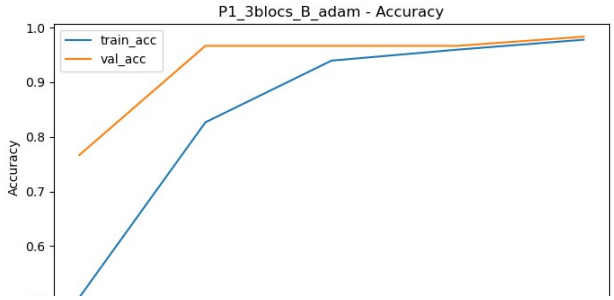

P1_ep5_A_adam - Loss
train_loss
val_loss
Loss
Epoch

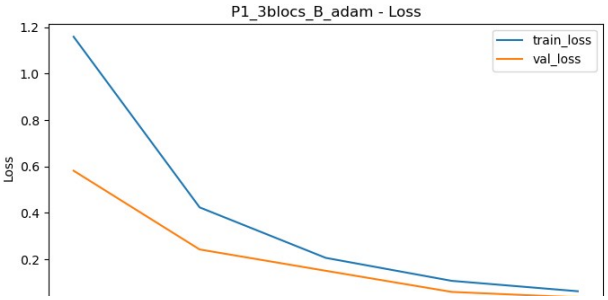

P1_ep10_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch


P1_ep10_A_adam - Loss
train_loss
val_loss
Loss
Epoch


P1_2blocs_A_adam - Accuracy
train_acc
val_acc
Accuracy
Epoch


P1_2blocs_A_adam - Loss
train_loss
val_loss
Loss
Epoch


P1_3blocs_B_adam - Accuracy
train_acc
val_acc
Accuracy


P1_3blocs_B_adam - Loss
train_loss
val_loss
Loss

```
# Matrice de confusion + classification report pour le meilleur modele
if DATA_DIR is not None and len(all_results) > 0:
    best_exp_name =
pd.DataFrame(all_results).sort_values('best_val_acc',
ascending=False).iloc[0]['name']
    best_model = all_models[best_exp_name]

    # Reconstruit un test_ds avec le meme batch_size que l'experience
gagnante
    best_exp = next(x for x in EXPERIMENTS if x['name'] ==
best_exp_name)
    _, _, best_test_ds, best_class_names = make_datasets(DATA_DIR,
batch_size=best_exp['batch_size'])

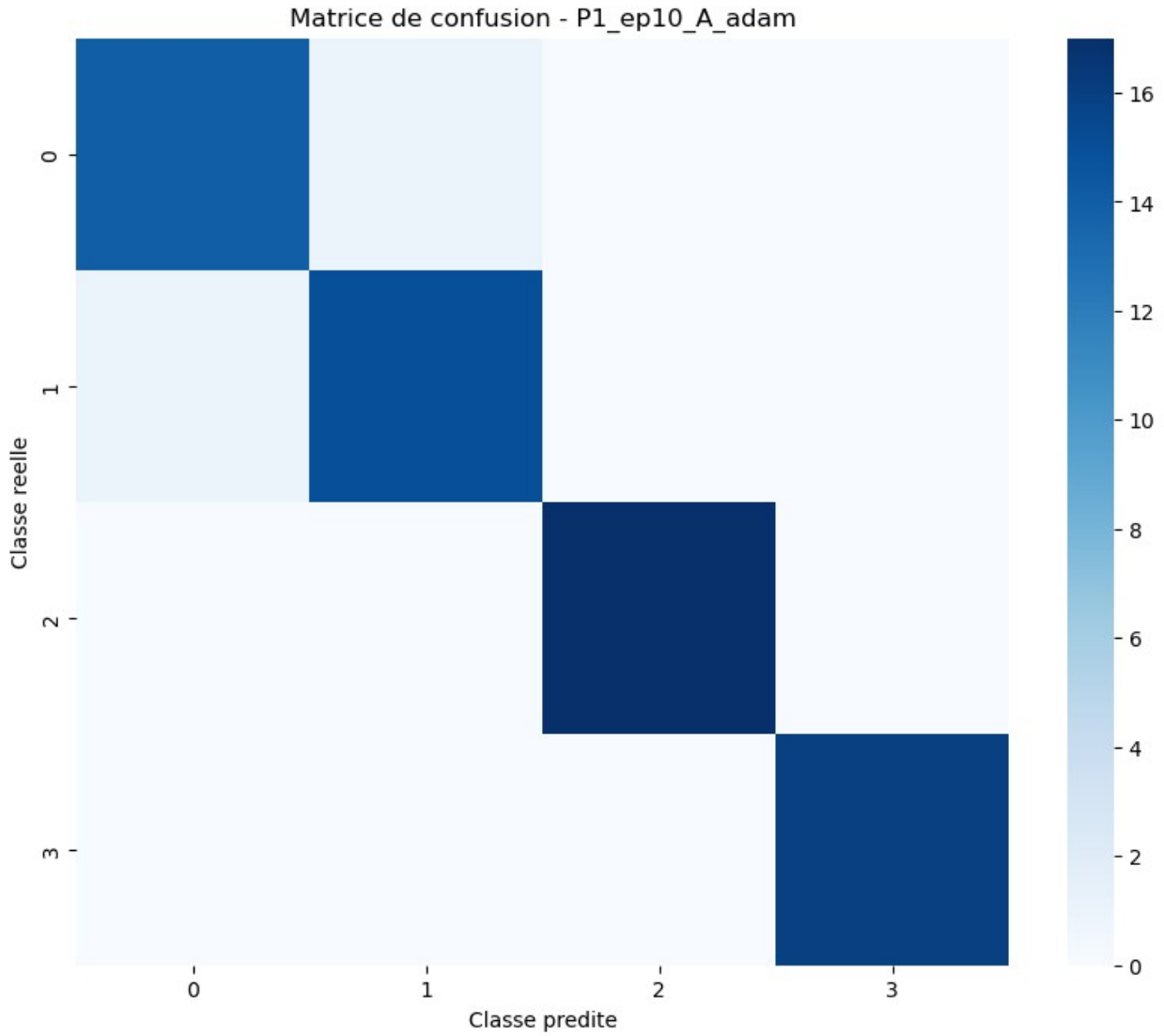
    y_true = []
    y_pred = []

    for images, labels in best_test_ds:
        preds = best_model.predict(images, verbose=0)
        y_true.extend(labels.numpy().tolist())
        y_pred.extend(np.argmax(preds, axis=1).tolist())

    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=False, cmap='Blues')
    plt.title(f'Matrice de confusion - {best_exp_name}')
    plt.xlabel('Classe predite')
    plt.ylabel('Classe reelle')
    plt.show()

    print('Meilleure experience:', best_exp_name)
    print(classification_report(y_true, y_pred,
target_names=best_class_names, digits=4))
```

```
Found 620 files belonging to 4 classes.
Using 496 files for training.
Found 620 files belonging to 4 classes.
Using 124 files for validation.
```




```
Meilleure experience: P1_ep10_A_adam
              precision    recall  f1-score   support

bitter orange     0.9333    0.9333    0.9333        15
         rose     0.9375    0.9375    0.9375        16
 sweet potato     1.0000    1.0000    1.0000        17
 vieux garcon     1.0000    1.0000    1.0000        16

     accuracy                         0.9688        64
    macro avg     0.9677    0.9677    0.9677        64
 weighted avg     0.9688    0.9688    0.9688        64
```